

Assignment 1 – Performance Requirements & Testing

This assignment is worth 15% of your final mark. Final submissions are due 2359 Friday 9 October 2026 (Week 6).

In this assignment, you will build a system known as the "system under test", establish performance and accuracy requirements for it, and design and execute tests that determine whether or not the system meets those requirements. Your work ends in a recommendation to a client. The requirements, instrumentation, golden test set, and baseline measurements from this assignment will form the basis of optimisation and performance tuning in your second assignment.

This assignment is to be completed in teams of four or five students. Please form your own teams and register them on xSiTe by the end of Week 2 (Friday 11 September 2026). All members of a team must be from the same lab group.

Client Scenario

Your client is a financial services company whose customer relations desk receives a steady stream of complaint tickets. Today, every ticket is read and routed by a human. The client wants incoming tickets classified automatically by category, so that they can be routed to the right team.

The client's constraint: no public model API may be used. Ticket narratives contain sensitive customer financial information, and the client's compliance rules do not permit this data to leave their infrastructure. All model inference must run on hardware the client controls, and the client's available hardware is commodity CPU servers with no GPUs. Your engagement is therefore a constraint problem. The question is not "which model is best". The question is: given these constraints, what should the client deploy, and what service quality can you promise?

AI coding tools are permitted and expected throughout. The build is a few hours' work with an agent, and that is accepted. AI-generated code is typically correct but performance-naïve, and an agent will not label your test data, run your load tests, or make your recommendation. The assessment targets measurement, interpretation, and judgement.

System Under Test

The System Under Test consists of the following components:

1. **Ticket Triage Service.** A web service you build and run in Docker. It must expose, at minimum: POST /tickets, which accepts one ticket narrative in the request body, classifies it into one of the seven categories below by calling the model backend, stores the result, and returns the assigned category; GET /search, which returns stored tickets matching a text query; and GET /stats, which returns counts of stored tickets by category. The service starts empty. Tickets enter the system only through POST /tickets; there is no bulk import.
2. **Model backend.** Ollama, running on CPU only, on hardware your team controls. The choice of models is yours and is part of the engagement: select three to five candidate models from the Ollama library, spanning at least two parameter size classes, and justify the candidate set in your report. Pin each candidate by its exact Ollama tag and digest, and report the same pins with your results. GPU inference is not permitted, in keeping with the client constraint; report your hardware in the test environment description.

3. **Dataset.** A course extract from the Consumer Complaint Database published by the US Consumer Financial Protection Bureau: real consumer complaint narratives, published with consumer consent and with personal information removed at source (original database: www.consumerfinance.gov/data-research/consumer-complaints/). Tickets are classified into seven categories: Credit reporting, Debt collection, Mortgage, Credit card, Bank account or service, Consumer loan, and Money transfer or service. The extract is released on xSiTe in Week 1 as a single numbered CSV. Your team uses the 1,000 records from row $(n \times 1000)$ to row $(n \times 1000 + 999)$, where n is your registered team number; for example, team 4 uses rows 4000 to 4999. All labelling and all test traffic come from your team's rows. The category labels in the raw data were selected by consumers at submission time and are noisy; this is why you will build a golden test set in Step 1 rather than trusting the labels.
4. **Instrumentation.** Logging of every request handled by your service. Keep these logs and your JMeter result files in your repository for every run you report. Every number in your final document must reconcile with them; a number that cannot be traced to a log entry is treated as unsupported, and you may be asked to produce and explain your logs.

What You Will Do

The assignment is six steps. Steps 1 to 3 can proceed in parallel from Week 1. Step 5 cannot begin until Steps 1 to 4 are complete: your golden test set and your prediction record must both be committed to your repository before your first benchmark run (see Deliverables).

1. Build the golden test set.
2. Build the baseline service.
3. Model the client's workload.
4. Choose candidate models, set requirements, and record your predictions.
5. Test and measure.
6. Make your recommendation.

Step 1: Build the Golden Test Set

Classification accuracy cannot be measured against the raw dataset labels, because those labels are noisy. Your team will therefore construct a golden test set of 150 to 200 tickets drawn from your team's rows. This is deliberately the first step: it requires no code, labelling the data teaches you the categories before you write a single prompt, and the golden set must be finished before any model sees it. Labelling after you have seen model outputs drags your labels towards whatever the model says, and quietly corrupts the accuracy measurement.

1. Write a labelling protocol first: a definition of each category, with rules for the edge cases you expect (tickets that fit two categories, tickets that fit none, ambiguous narratives).
2. At least two team members label every ticket independently, following the protocol and without conferring.
3. Compute and report an inter-annotator agreement statistic.
4. Resolve every disagreement by discussion, record each resolution, and update the protocol where a disagreement revealed a gap in it.

The finished golden set is frozen before any model sees it: it is committed to your repository before your first benchmark run, and the commit history is your evidence (see Deliverables). Disagreements are

expected and are evidence of care, not error. A golden set with implausibly perfect agreement and no recorded resolutions will be examined closely.

Step 2: Build the Baseline Service

Build the service described above. In the Assignment 1 baseline, classification is synchronous: POST /tickets does not return until the model has classified the ticket. Beyond meeting the specification, the baseline should be the straightforward thing an agent writes: sequential calls, no caching, no queuing. Do not pre-optimize it. The baseline exists to be measured, and optimization is Assignment 2.

Note how data reaches the service: it arrives one ticket at a time through POST /tickets, exactly as it would from the client's complaint intake. During testing, your load generator plays the role of the intake, drawing ticket narratives from your team's rows and posting them to the service (Step 5). The dataset CSV is never loaded into the service directly.

Step 3: Model the Client's Workload

Your requirements must be informed by a quantitative workload model that estimates: the number of tickets the client receives in a relevant period; the rate of agent-side searches; peak and non-peak periods, if they exist; and the expected distribution of ticket lengths. Use publicly available figures relevant to a financial services complaints desk as much as possible, for example published complaint volume statistics, and cite the source for each figure. If no figure is available, make the best estimate you can and outline how you estimated it.

Step 4: Choose Candidate Models, Set Requirements, and Record Your Predictions

Choose your candidate models as described in the System Under Test section: three to five, spanning at least two size classes, each pinned by tag and digest, with the candidate set justified. Smaller models are faster on CPU and wrong more often; larger models are more accurate, slower, and sustain less throughput. Your candidates should make that trade-off visible rather than avoid it.

Set at least one requirement for each of the following, justified by your workload model together with other relevant considerations such as usability, the staffing cost of misrouted tickets, and capacity. If your model implies peak periods, your requirements must cater for the peak. Requirements must be testable: a number, a percentile where relevant, and the load condition under which it must hold.

- At least one response time requirement, for example the latency of POST /tickets, or of GET /search under mixed load.
- At least one throughput requirement, for example tickets classified per hour at sustained load.
- A classification accuracy requirement, overall and per category, to be measured on your golden test set.

Then write your prediction record. It is committed to your repository together with your golden set before your first benchmark run, and cannot be revised afterwards (see Deliverables). It must state, specifically enough to be provably wrong:

- Where you expect the bottleneck to be under load, and why.
- For each candidate model: expected classification accuracy on your golden set, and expected single-request latency on your hardware.
- Which categories you expect to be hardest to classify, and why.

Marks are awarded for specificity and for the quality of your later account of where your predictions were wrong, not for being right. A vague prediction that cannot fail earns nothing.

Step 5: Test and Measure

First, describe your test environment: the machines running the service, Ollama, and the load generator; their CPU, memory, and operating system; the network between them; and any factors that could make your measurements unrepresentative. Indicate how results from your test environment scale to the client's deployment. The load generator and the system under test must run on separate machines. A co-hosted load generator steals CPU from the service and produces latency numbers that are fiction.

Then run three kinds of test.

Load tests, using Apache JMeter. JMeter plays the role of the client's complaint intake: it draws ticket narratives from your team's rows (for example with a CSV Data Set Config) and posts each one to POST /tickets. The protocol is fixed so that results are comparable:

- Traffic must be generated open-loop, at controlled arrival rates: use the Open Model Thread Group or the Precise Throughput Timer. Closed-loop traffic self-throttles when the server slows down and hides queue buildup. Results from closed-loop tests will not be accepted as evidence against throughput or latency requirements.
- Report at minimum p50, p95, and p99 latency, achieved throughput, and error rate, at each tested arrival rate.
- Three runs per configuration. Report means and the spread across runs. A single run is not a measurement.
- Keep the raw JMeter result files (.jtl) from every run in your repository; they must reconcile with your service logs.

Accuracy tests. Send every golden-set ticket through POST /tickets for each candidate model, and report overall and per-category accuracy against your golden labels, with a confusion matrix.

One stress test. Design and execute one test that determines a limit of the system under test, for at least one candidate model. For example, you might determine the maximum ticket arrival rate sustainable before latency grows without bound, or the number of concurrent requests at which the service or Ollama begins to fail, or any other meaningful limit of the system.

Each test playbook must be described in sufficient detail that a competent software tester could carry it out without seeking or inventing further information from your team. Include the results of every test when applied to the unmodified baseline. Note any instance where the system does not meet your requirements; you do not need to correct the root cause at this stage. Diagnose it and note that it exists.

Step 6: Make Your Recommendation

The engagement ends in a recommendation, not a benchmark table. Given the client's constraints, recommend which of your candidate models the client should deploy. The candidates will not agree, and neither this brief nor the client tells you whether a misrouted ticket or a slow triage costs the client more. Your workload model and requirements must take a position, and your recommendation must follow from your own measurements and hold against your own requirements.

A recommendation that contradicts your stated requirements, or rests on numbers that do not appear in your logs, fails regardless of which model it picks. A finding that no candidate meets all of your

requirements, measured carefully and argued clearly, is a strong result; consultants deliver that finding to clients regularly.

Deliverables

Submission Format

- One PowerPoint file, maximum 12 slides, plus the supporting files listed below.
- Submit via xSiTe by 2359 Friday 9 October 2026 (Week 6).
- File name format: GroupNum.pptx, e.g. Group01.pptx.
- Supporting files, in the same submission: the golden test set (final labels for the 150 to 200 tickets, identified by row number); the prediction record (the three items listed in Step 4); and the labelling protocol with its revisions, the independent label sheets, and the agreement statistic.

The golden test set and the prediction record must also be committed to your repository before your first benchmark run. The commit history is your evidence that your labels and predictions predate your measurements. Keep your raw JMeter .jtl files and service logs in the repository as well; you will not submit them, but every number in your document must reconcile with them, and you may be asked to produce and explain them.

Slide-by-Slide Requirements

Slide 1 – Cover Page

- Group number, names, and student IDs.
- Project title (concise and descriptive).
- Link to your team's GitHub repository, containing the service source code, sufficient to rebuild and run it with docker compose.

Slide 2 – Service Architecture

- Diagram of the components: triage service, Ollama backend, storage, load generator.
- The three endpoints and what each does.
- Confirmation that the baseline is synchronous, with no caching or queuing.

Slide 3 – Workload Model

- Estimated ticket volumes, search rates, and peak versus non-peak periods.
- Expected distribution of ticket lengths.
- Cite the source of every figure; state clearly which figures are estimates and how you estimated them (see Slide 12).

Slide 4 – Performance and Accuracy Requirements

- The response time, throughput, and accuracy requirements, each as a testable statement: a number, a percentile where relevant, and the load condition under which it must hold.
- Justify each requirement from your workload model and other relevant considerations.

Slide 5 – Candidate Models

- Your three to five candidate models, each pinned by Ollama tag and digest.
- The size classes they span and the justification for the candidate set.

Slide 6 – Golden Test Set

- Summary of the labelling protocol and its revisions.

- The inter-annotator agreement statistic.
- Number of disagreements and how they were resolved, with one or two examples.

Slide 7 – Test Environment

- Hardware and software of each machine: service, Ollama, load generator.
- Confirmation that the load generator ran on a separate machine.
- How results scale to the client's deployment; assumptions and limitations.

Slide 8 – Playbook (Testing Procedure)

- Step-by-step outline of how each test was conducted, including the open-loop JMeter configuration.
- Include diagrams or flowcharts where useful.

Slide 9 – Load and Stress Test Results

- p50, p95, and p99 latency, achieved throughput, and error rate at each tested arrival rate, per candidate model, across three runs.
- The stress test and the limit it found.
- Short interpretation, including the diagnosed bottleneck.

Slide 10 – Accuracy Results

- Overall and per-category accuracy on the golden set, per candidate model.
- Confusion matrix highlights: where each model goes wrong.

Slide 11 – Predictions, Recommendation and Defence

- Your predictions against your outcomes, and an account of where and why you were wrong.
- The recommended model, defended against your stated requirements.
- Any requirement no candidate meets, stated plainly.

Slide 12 – References and Acknowledgements

- Properly formatted references for all sources used, including the Consumer Complaint Database, Ollama, and the licences of your candidate models.

Late submissions will be penalised at 15% per day. No submissions will be accepted more than four days after the due date.

Assessment Criteria

Submissions will be assessed for satisfaction of the assignment requirements described above; the completeness and plausibility of the workload model; conformance of requirements to that model; the rigour of the golden set construction; the quality of the test procedures and the completeness of the test results; the defensibility of the recommendation; and the logical structure, clarity and presentation of the report. The 15% is allocated as follows:

Component	What is assessed	Weight
Implementation (Step 2)	Working triage service meeting the specification; logs kept in the repository and reconcilable with reported numbers	5%
Requirements and golden test set (Steps 1, 3 and 4)	Golden set built to a written protocol with independent labelling and an	5%

	agreement statistic, and frozen on time; workload model complete and plausible; requirements derived from it; candidate model set justified; prediction record specific enough to be wrong	
Measurement and recommendation (Steps 5 and 6)	Three-run JMeter results; accuracy results per candidate model on the golden set; bottleneck identified; stress test executed; recommendation defended against the stated requirements; every number reconciles with the kept logs	5%

Notes on Copyright and Plagiarism

This assignment uses data and software that already exist. The Consumer Complaint Database is published by a US government agency; acknowledge the source in your final document. Ollama and your candidate models each carry their own licences; place any acknowledgements they require in your final document. Failing to comply with a licence may be an infringement of copyright.

The University's policy on copying does not allow you to copy your assessment solutions from another person or team. AI coding tools are permitted; copying another team's work is not, and your data rows, hardware, candidate models, and golden set differ from theirs in any case. It is the responsibility of all students that their assessment solutions are their own work, and that others do not obtain access to their solutions for the purpose of copying. Where plagiarism is detected, both of the assessments involved will receive ZERO mark. Fabricated or irreconcilable measurement numbers are treated as an academic integrity matter, not a marking deduction.